

IS examen

Estructura Mental: Las 4 Capas de Anotaciones

Para no confundirte, divide las anotaciones en tu cabeza según el archivo en el que las vas a escribir:

```
mermaid
```

```
graph TD
    A[Tus Archivos Java] --> B[1. Lombok: Ahorrar Código]
    A --> C[2. JPA: Base de Datos]
    A --> D[3. Validation: Proteger Datos]
    A --> E[4. Spring: Arquitectura y API]
```

1. Lombok (Anotaciones para tus clases de negocio: Entidades y DTOs)

Estas son siempre las mismas y van sobre la definición de la clase. Te ahorran escribir getters, setters y constructores manualmente.

- `@Getter` y `@Setter`: Generan los métodos de acceso.
- `@NoArgsConstructor`: Constructor vacío obligatoria para JPA.
- `@AllArgsConstructor`: Constructor con todos los campos (requerido por `@Builder`).
- `@Builder`: Te permite crear objetos de forma fluida (ej: `Paciente.builder().nombre("Juan").build()`). Es sumamente útil en la lógica del Service y en los tests.

2. JPA / Jakarta Persistence (Para mapear tus Entidades del paquete `domain` a SQL)

Aquí es donde el examen te pedirá reglas de base de datos. Aprende a traducir la frase del examen a la anotación:

Si el examen te dice...	La anotación en Java es...	Ejemplo de uso
"Es una entidad / tabla en la base de datos"	<code>@Entity</code> <code>@Table(name = "nombre_plural")</code>	<code>@Entity</code> <code>@Table(name = "pacientes")</code>
"Es la clave primaria autoincremental"	<code>@Id</code> <code>@GeneratedValue(strategy = GenerationType.IDENTITY)</code>	Sobre el atributo <code>Long id</code> ;
"Este campo no puede ser nulo en SQL"	<code>@Column(nullable = false)</code>	<code>@Column(nullable = false)</code>
"Debe tener una longitud máxima de X"	<code>@Column(length = X)</code>	<code>@Column(length = 100)</code>
"Este campo es único (ej: DNI o Email)"	<code>@Column(unique = true)</code>	<code>@Column(unique = true)</code>
"Manejar precios o montos de dinero"	<code>@Column(precision = 10, scale = 2)</code> (Recuerda usar tipo <code>BigDecimal</code>)	<code>@Column(precision = 10, scale = 2)</code>
"Es un estado/enum en formato de texto"	<code>@Enumerated(EnumType.STRING)</code>	En tu enum de estado (ej: <code>EstadoCita</code>)

"Muchos [A] pertenecen a un [B]" (ej: Muchas Citas pertenecen a 1 Paciente)	<code>@ManyToOne(fetch = FetchType.LAZY)</code> <code>@JoinColumn(name = "b_id", nullable = false)</code>	<code>@ManyToOne(fetch = FetchType.LAZY)</code> <code>@JoinColumn(name = "paciente_id")</code>
"Un [B] tiene muchos [A]" (ej: Un Paciente tiene muchas Citas)	<code>@OneToMany(mappedBy = "b", cascade = CascadeType.ALL, orphanRemoval = true)</code>	<code>@OneToMany(mappedBy = "paciente")</code> <code>private List<Cita> citas;</code>

IMPORTANT
Regla de oro para relaciones en exámenes: Pon siempre `FetchType.LAZY` en los `@ManyToOne` y `@OneToOne`. Esto le demuestra al evaluador que te importa el rendimiento (evita el problema de consultas N+1).

3. Jakarta Bean Validation (Para validar datos entrantes en tus DTOs / Records)

Estas anotaciones van en tus DTOs (como `ProductoRequest` o `CitaRequest`) para asegurar que el cliente no envíe datos basura. El controlador las activa usando la anotación `@Valid` en el parámetro del método.

Si el requerimiento del examen dice...	La anotación en el DTO/Record es...
"El texto no debe estar en blanco ni ser nulo"	<code>@NotBlank(message = "...")</code> (Exclusivo para Strings)
"El valor es requerido (cualquier tipo)"	<code>@NotNull(message = "...")</code> (Para números, fechas, objetos, etc.)
"Debe ser un correo electrónico válido"	<code>@Email(message = "...")</code>
"Debe ser un número positivo (ej: precios o stock)"	<code>@Positive(message = "...")</code> o <code>@PositiveOrZero</code>
"El número mínimo debe ser X"	<code>@Min(value = X)</code>
"La fecha debe ser en el futuro (ej: reservar cita)"	<code>@Future(message = "...")</code> o <code>@FutureOrPresent</code>
"La fecha debe ser en el pasado (ej: fecha de nacimiento)"	<code>@Past(message = "...")</code>

4. Spring Core y Spring Web (Para la arquitectura del sistema)

Estas le dicen a Spring qué rol juega cada clase en el contenedor de inversión de control (IoC):

- **Capa Datos:** `@Repository` en tus interfaces que heredan de `JpaRepository`.
- **Capa Negocio:** `@Service` en tus clases de servicio.
- **Capa API:** `@RestController` y `@RequestMapping("/api/v1/recurso")` en tus controladores.
- **Métodos HTTP:** `@GetMapping`, `@PostMapping`, `@PutMapping`, `@DeleteMapping`.
- **Parámetros en API:**
 - `@PathVariable`: Para leer variables de la URL (ej: `/pacientes/{id}`).
 - `@RequestBody`: Para recibir el JSON en peticiones POST/PUT.
 - `@RequestParam`: Para filtros opcionales en URL (ej: `/pacientes?nombre=Juan`).
- **Validación en API:** `@Valid` en el cuerpo del controlador (ej: `(@Valid @RequestBody CitaRequest request)`).
- **Transacciones:** `@Transactional` para operaciones de escritura en la base de datos (Service), y `@Transactional(readOnly = true)` para búsquedas GET.

Estrategia del Examen: Cómo "traducir" el caso al código

Cuando leas el caso de prueba en tu examen, sigue este flujo mental para mapear requerimientos a anotaciones:

1. Paso 1: Identificar las entidades principales e intermedias.

- *Ejemplo:* Si te dicen "Se desea registrar la asistencia de los alumnos a un curso".
- *Entidades:* `Alumno` , `Curso` y una tabla intermedia/relación `Asistencia` .

2. Paso 2: Escribir la estructura básica JPA y Lombok.

- Agrega `@Entity` , `@Table` , `@Getter` , `@Setter` , `@NoArgsConstructor` , `@AllArgsConstructor` , `@Builder` a todas las entidades del paquete `domain` .

3. Paso 3: Leer las restricciones del caso de uso una a una.

- *Caso dice: "El código del alumno es único y obligatorio."*
 - 👉 Traduces a la Entidad: `@Column(unique = true, nullable = false)` .
 - 👉 Traduces al Request DTO: `@NotBlank` o `@NotNull` .
- *Caso dice: "Un alumno puede registrar múltiples asistencias."*
 - 👉 Traduces a la Entidad `Asistencia` : Relación `@ManyToOne(fetch = FetchType.LAZY)` apuntando a `Alumno` .
- *Caso dice: "La fecha de asistencia debe ser obligatoriamente la fecha actual o futura."*
 - 👉 Traduces al DTO de entrada: `@FutureOrPresent` .